
Spatial Atlas: Compute-Grounded Reasoning for Spatial-Aware Research Agent Benchmarks

Arun Sharma

University of Minnesota, Twin Cities
arunshar@umn.edu

Abstract

We introduce *compute-grounded reasoning* (CGR), a design paradigm for spatial-aware research agents in which answerable sub-problems are computed from explicit intermediate representations before a language model generates a response. Spatial Atlas instantiates CGR as an Agent-to-Agent (A2A) server with spatial question-answering and machine-learning engineering handlers. A structured spatial scene graph computes relationships from extracted entities. A strict metric perception bridge replaces model-estimated coordinates with geometry reconstructed from segmentation masks and depth, and it fails closed rather than substituting estimated coordinates when its evidence checks do not pass. The system also provides the run modes and validators for a four-path label-free comparison, covering a scene-graph path, a question-only baseline, a correct-image metric path, and a shuffled-image metric control. The ML path supports strategy-aware code generation, validation-score parsing, bounded refinement, and leak-audit prompts, but generated-code execution is disabled by default and requires explicit opt-in inside an isolated, trusted worker. We report one completed label-free operational validation in which all four paths executed and closed their bounded lifecycle with protected labels and ground truth sealed, which establishes operational integrity rather than benchmark accuracy or comparative performance. This paper describes the implemented architecture, that operational validation, and the protected scoring protocol that remains pending. It does not report FieldWorkArena results because the benchmark data were not accessible, and it intentionally omits performance, cost, and latency numbers that are not backed by reproducible run artifacts.

1 Introduction

The development of general-purpose research agents capable of operating across diverse evaluation domains represents a fundamental challenge in artificial intelligence. While large language models (LLMs) have demonstrated remarkable reasoning capabilities [1, 23], deploying them as autonomous agents that can reliably solve real-world tasks remains an open problem [26]. Two recent benchmarks highlight complementary dimensions of this challenge: FieldWorkArena [9], which evaluates multimodal spatial reasoning in industrial environments such as factories, warehouses, and retail spaces, and MLE-Bench [4], which tests end-to-end machine learning engineering across 75 Kaggle competitions.

Most existing agent architectures treat these benchmarks as independent problems, developing specialized systems for each [13, 30]. This fragmentation wastes shared infrastructure and misses opportunities for architectural insights that transfer across domains. For instance, the structured reasoning required to answer spatial questions (“How many pallets are within 3 meters of the emergency exit?”) shares fundamental properties with the systematic hypothesis testing needed to select effective ML strategies (“Which feature engineering approach maximizes validation accuracy for this tabular dataset?”).

We present **Spatial Atlas**, a spatial-aware research agent that exposes spatial question-answering and ML-engineering handlers through a single Agent-to-Agent (A2A) protocol server [10]. Field-

WorkArena motivated the original spatial adapter, but the benchmark is not part of the reported evaluation because its data were not accessible. The system is organized around a design paradigm we call *compute-grounded reasoning* (CGR): wherever a sub-problem admits a deterministic solution, compute the answer first and supply it as a fact to the language model rather than asking the model to generate it. Our architecture instantiates CGR through five key contributions:

1. **Spatial Scene Graph Engine:** A structured representation that extracts entities and relations from vision model descriptions, computes spatial relationships, and places the computed facts in the prompt. It does not record whether a stored distance came from the model or from code. Perception errors can still propagate into the graph.
2. **Confidence-Gated Refinement:** One fast-tier self-grade decides whether the Strong tier writes at most one refinement. The codebase also defines an expected-information-gain action selector, but the spatial path never calls it. The accuracy and cost effects of both remain evaluation questions.
3. **Fail-Closed ML Pipeline:** A strategy-aware code generation system whose execution path requires execution opt-in, isolated-worker attestation, and authenticated server startup. An enabled run permits at most 3 total attempts with a 600-second timeout per attempt, and dummy submissions require a separate opt-in.
4. **Score-Driven Refinement:** An iterative loop that parses machine-readable validation scores, asks the configured strong tier for a revision, and retains the revision only when its parsed score is better.
5. **Leak Audit Registry:** A prompt-based framework that asks generated pipelines to check common train/test leakage patterns and can inject task-specific hints.

The unifying principle behind these contributions is compute-grounded reasoning: wherever possible, we compute answers from structured representations rather than asking language models to generate them directly. This design improves inspectability because intermediate inputs and computations can be audited. Reliability, accuracy, latency, and cost must still be established through the planned evaluation.

2 Related Work

Agent Frameworks: The rapid development of LLM-based agent frameworks has produced systems spanning general-purpose reasoning and specialized domains. AutoGPT [25] pioneered autonomous LLM agents with self-directed task decomposition, while OpenDevin (now OpenHands) [13] established a software development agent framework with sandboxed code execution. SWE-Bench agents [15] demonstrated that LLMs can resolve real-world GitHub issues, and DAMO MLE-Agent [31] specifically targets Kaggle-style ML competitions. Our work differs in unifying two distinct benchmark domains under a single architecture with shared compute-grounded reasoning infrastructure.

Spatial Reasoning in Vision-Language Models: Vision-language models (VLMs) exhibit well-documented weaknesses in spatial reasoning tasks, particularly object counting, distance estimation, and relative positioning [5, 22]. Studies have shown that VLMs frequently hallucinate spatial relationships when asked to reason about complex scenes [18]. SpatialVLM [5] attempts to address this through specialized spatial training data. Our approach instead moves relationship computation into an explicit representation, while remaining dependent on the accuracy of entity extraction and geometric measurement.

Scene Graphs for Visual Reasoning: Scene graph representations, popularized by Visual Genome [17] and the GQA dataset [14], provide structured representations of visual scenes as graphs of objects and relationships. Neural scene graph generation [29] and scene graph-based visual question answering [11] have shown that explicit structure improves reasoning over raw visual features. Our spatial scene graph engine adapts these ideas to industrial environments, incorporating distance computation and constraint checking as first-class operations.

AutoML and Competition-Oriented Systems: Automated machine learning frameworks such as AutoGluon [7], Auto-sklearn [8], and AutoKeras [16] aim to automate the end-to-end ML pipeline. More recent work leverages LLMs for ML code generation [12], combining the flexibility of natural language understanding with systematic hyperparameter search. Our ML path adds strategy-aware code generation and bounded repair attempts behind a fail-closed execution gate.

A2A Protocol and Agent Interoperability: Google’s Agent-to-Agent (A2A) protocol [10] defines a standard for inter-agent communication, enabling heterogeneous agents to collaborate through a common interface. Our system implements a compliant A2A server that exposes both spatial

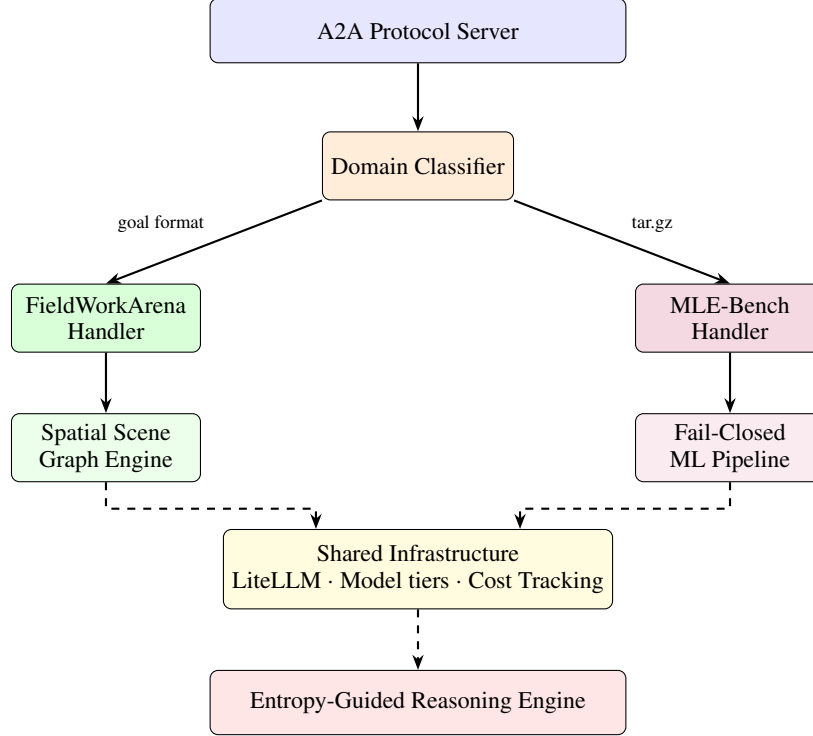


Figure 1: Spatial Atlas system architecture. The A2A server routes incoming tasks to domain-specific handlers through a classifier. Both handlers share the LiteLLM client, the configured model tiers, and usage tracking. Only the FieldWork handler calls the entropy engine, and it uses the engine only for its self-grade.

reasoning and ML pipeline capabilities through a unified task interface, which exercises the protocol for multi-domain agent deployment.

Information-Theoretic Reasoning: Active learning [24] and Bayesian experimental design [3] provide principled frameworks for selecting actions that maximize information gain. Recent work has applied these ideas to LLM reasoning chains [28], using uncertainty estimates to guide when to seek additional information. Our design proposes an entropy-guided extension of this paradigm to agent action selection, estimating which reasoning step would most reduce uncertainty about the final answer. This repository does not call that selector, so no result here shows its effect.

3 System Architecture

Spatial Atlas operates as a spatial-aware research agent exposed via a dual-domain A2A server. It receives task requests through a standardized protocol and routes them to the appropriate processing pipeline. Figure 1 illustrates the overall system design.

Domain Classification: The domain classifier operates on task metadata and attachment types. The FieldWorkArena adapter recognizes its documented goal shape, while MLE-Bench tasks arrive with `tar.gz` attachments containing competition datasets and description files. Classification uses deterministic rules rather than an LLM call. The implementation has not been benchmarked for routing latency or cost.

3.1 Shared Infrastructure

Both domain handlers share several critical infrastructure components:

LiteLLM Multi-Provider Wrapper: We use LiteLLM [21] to abstract across multiple LLM providers and record provider-reported usage when available. Provider reports and local estimates are not treated as exact tokenizer-equivalent counts.

Three-Tier Frontier Model Routing: We define three model tiers, *fast*, *standard*, and *strong*, each with a distinct intended role, as shown in Table 1. The tiers are roles rather than fixed ven-

Table 1: Configured model tiers and intended roles. This is a design table, not a performance or cost result.

Tier	Model	Intended role
Fast	GPT-4.1-mini	Entity extraction and self-confidence scoring
Standard	GPT-4.1	MLE competition analysis
Strong	GPT-4.1	Spatial reasoning, code generation, refinement

dors. In the default configuration the standard and strong tiers resolve to the same model, and an operator may point the strong tier at a different provider. Section 5 states which tiers the spatial path actually invokes and in what order.

The default configuration maps Standard and Strong to GPT-4.1. Operators can override `ATLAS_STRONG_MODEL` to test a different provider. Same-model and cross-provider refinement remain planned ablations. No accuracy or cost result is claimed here.

Public Agent Token Reservation: The public A2A *Agent* uses a concurrency-safe budgeted client governed by a legacy per-execution reservation ceiling, configured by default at 150,000 tokens. That figure is a configured cap, not a measurement of consumption, and this paper reports no observed token usage. Before each provider call, it heuristically estimates prompt usage and reserves that estimate plus the allowed maximum completion under a lock. Concurrent calls within one A2A execution cannot oversubscribe the estimated reservation counter. A new execution receives a fresh counter, even for the same A2A task ID. This is not exact tokenizer accounting and not a hard provider-token boundary, because provider tokenizers and image accounting vary. Provider-reported usage remains authoritative. Frozen benchmark drivers retain the base observational client. Their batch usage is reconciled from immutable journals, terminal counters, and result artifacts.

4 Spatial Scene Graph Engine

The spatial scene graph engine is the cornerstone of the spatial question-answering path and the unexecuted FieldWorkArena adapter. It is designed to make spatial computations explicit, but it does not remove errors introduced by perception, depth estimation, object matching, or coordinate assumptions.

4.1 Problem Formulation

Given an image I of an industrial environment (factory, warehouse, or retail space) and a natural language question q , the task is to produce an answer a that may require counting objects, estimating distances, checking spatial containment, or verifying safety compliance. Directly prompting a VLM with (I, q) is unreliable because VLMs hallucinate spatial relationships and struggle with precise counting.

4.2 Scene Graph Construction

Our approach decomposes the problem into three stages: *extraction*, *structuring*, and *computation*.

Stage 1: Entity Extraction: We employ a two-pass extraction process. First, a vision-language model (GPT-4.1 with vision) generates a detailed textual description of the scene, prompted to enumerate all visible objects with approximate positions and attributes. Second, Florence-2 [27], a lightweight vision foundation model, performs object detection to obtain bounding boxes and counts, intended to ground the VLM’s descriptions. Its detection quality on industrial imagery has not been measured here.

Stage 2: Graph Construction: Extracted entities are formalized as a spatial scene graph $G = (V, E)$ where vertices V represent entities and edges E represent spatial relations:

$$v_i = \text{SpatialEntity}(\text{id}_i, \text{label}_i, \text{pos}_i, \text{attrs}_i, \text{zone}_i) \quad (1)$$

$$e_{ij} = \text{SpatialRelation}(\text{subj}_i, \text{pred}_{ij}, \text{obj}_j, d_{ij}) \quad (2)$$

where $\text{pos}_i \in \mathbb{R}^2$ denotes a position estimated by the vision-language model and returned in its structured extraction, not a measured or bounding-box-derived coordinate, attrs_i is a dictionary of

visual attributes (color, size, state), zone_i identifies the semantic zone (e.g., loading dock, aisle 3), and d_{ij} is the computed Euclidean distance between entities.

Stage 3: Deterministic Computation: The scene graph supports several query operations that produce verifiable facts:

- `query_near( $v$ ,  $r$ )`: Returns all entities within radius r of entity v .
- `check_constraints( $C$ )`: Evaluates a set of spatial constraints C (e.g., minimum clearance distances) and returns violations.
- `to_fact_sheet()`: Serializes the graph into a structured natural language summary suitable for LLM consumption.

The fact sheet is then provided to the LLM alongside the original question, enabling it to answer based on computed facts rather than visual estimation.

4.3 Metric Perception Bridge

The scene graph described above performs repeatable arithmetic over *model-estimated* two-dimensional coordinates. The arithmetic is deterministic, but its inputs are not measurements. The metric perception bridge addresses that gap by replacing estimated coordinates with geometry derived from segmentation masks and a reconstructed metric point map.

For a frozen surface-gap question over two referenced objects, the bridge proceeds in fixed steps. Its strict parser targets the horizontal-gap questions of Q-Spatial Bench [19]. Each object mask is eroded with a scale-aware one-pass operator whose radius is $\lceil \max(w_m/w_t, h_m/h_t) \rceil$ pixels, where (w_m, h_m) is the mask resolution and (w_t, h_t) the reconstruction target resolution, using a closed Euclidean disk structuring element. Surviving pixels are restricted to finite, confidence-qualified reconstruction points. Those points are then quantized onto a horizontal ground plane by the voxel key $(\lfloor x/0.005 \rfloor, \lfloor z/0.005 \rfloor)$, that is, at a five-millimetre resolution, and each occupied voxel contributes one deterministic representative chosen by highest reconstruction confidence with the lowest flat pixel index as a tie-break. When a mask exceeds the voxel cap, the retained subset is selected by a deterministic content hash rather than by sampling order, so the same inputs always yield the same representatives.

The surface gap between object sets A and B is then the symmetric minimum of two directed fifth-percentile nearest-neighbour distances in the horizontal plane:

$$g(A, B) = \min(q_{05}(\text{nn}_{xz}(A \rightarrow B)), q_{05}(\text{nn}_{xz}(B \rightarrow A))) \quad (3)$$

The fifth percentile rejects isolated outlier points without discarding the near-contact region that the question asks about, and taking the symmetric minimum removes the dependence on which object is named first.

Two properties of this path matter for interpretation. First, it is deliberately *fail-closed*: it records protocol identifiers, provenance, evidence digests, and range checks, and when any evidence check does not pass it returns an explicit unavailable result rather than silently substituting the estimated-coordinate scene graph. This differs from the general-purpose metric engine, which retains a documented graceful fallback to the scene-graph path. Second, reconstructed geometry is an estimate, not ground truth. Segmentation errors, reconstruction scale errors, and referent mismatches all remain possible, and the fail-closed checks bound how such failures surface rather than eliminating them. The bridge is public, but it is not a stand-alone reproduction. The live perception path requires an externally installed copy of NVIDIA’s SpatialClaw [6], whose GPU service runs SAM 3 segmentation [2] and Depth Anything 3 reconstruction [20]. Neither SpatialClaw nor its GPU perception service is vendored here, and the gated benchmark images and the frozen control artifact are not bundled with the repository.

4.4 Label-Free Run Modes and Controls

The repository provides the run modes and validators needed for a four-path comparison. Two axes select a path: a reasoning engine, either the scene-graph engine or the metric engine, and an image mode, one of correct image, question only, or shuffled image. The axes are not freely combinable. The question-only mode requires the scene-graph engine, because it withholds the image entirely and therefore has no geometry to measure. The shuffled-image mode requires the metric engine and an explicit control mapping, and a control mapping is rejected for any other image mode. Only four of the six combinations are admitted. Their combination yields the four paths used in Section 8: scene graph with the correct image, question only, metric with the correct image, and metric with a shuffled image.

Table 2: Scoring semantics used by the FieldWorkArena evaluating agent, which the local output formatter targets. These functions are benchmark-side and are not implemented in this repository. The thresholds shown are the benchmark’s documented defaults, not configuration in this system.

Metric	Description
fuzzy_match	Token-level overlap with configurable threshold (default 0.8)
exact_match	Case-insensitive exact string equality
must_include	Predicted answer must contain all specified substrings
must_exclude	Predicted answer must not contain any specified substrings
json_match	Structured comparison of JSON objects with field-level matching
numerical_match	Numeric comparison with a configurable relative tolerance, default 0.05

The shuffled-image control is contract-bound rather than ad hoc. The loader verifies the recorded digest of every control image and rejects the run if any image has drifted, and it rejects any mapping that is not a bijection or that contains a fixed point, since either defect would let an image be paired with itself and quietly weaken the control.

Prediction rows are written to append-only journals under a label-free schema. The writer rejects label-bearing fields anywhere in a payload, refuses to mix label-free and ordinary rows within one journal, and refuses to resume a journal whose schema does not match the current run. These validators enforce that a run cannot observe protected labels, which is what makes a pre-scoring execution check possible at all. What the public repository does not provide is the combined single-command orchestrator that runs all four paths under one root.

Output Formatting Against Benchmark Scorers: FieldWorkArena scoring is performed by the benchmark’s own evaluating agent, not by this repository. Spatial Atlas contains no implementation of these scoring functions. What it does implement is an output formatter that shapes each answer so a correct answer is not rejected on presentation, since a right answer in the wrong shape scores zero under exact and structured matchers. Table 2 therefore documents the benchmark-side scoring semantics that the formatter targets. Because the gated benchmark data were never accessible, this formatting has never been exercised against an official FieldWorkArena evaluation run.

5 Entropy-Guided Reasoning

This section states an information-theoretic formulation for selecting actions by expected information gain, drawing on active learning [24] and Bayesian experimental design [3]. We then state precisely which part of it the current system executes, because the two differ and the difference is material to how the design should be judged.

The formulation is given in Equations 4 and 5. The implemented controller is narrower. It produces one Strong-tier answer, obtains one fast-tier self-reported confidence score for that answer, and performs at most one Strong-tier refinement when the score falls below the threshold. General expected-information-gain action selection over a candidate set is present in the codebase as a component but is not invoked by the spatial reasoning path, so no claim in this paper depends on it. Cost effects of the routing policy are likewise unmeasured.

5.1 Information State Representation

At each reasoning step t , the agent maintains a knowledge state $\mathcal{K}_t$ consisting of accumulated observations, computed facts, and intermediate conclusions. We define the *answer entropy* as the uncertainty over the space of possible answers:

$$H(\mathcal{A} \mid \mathcal{K}_t) = - \sum_{a \in \mathcal{A}} P(a \mid \mathcal{K}_t) \log P(a \mid \mathcal{K}_t) \quad (4)$$

where $\mathcal{A}$ is the set of candidate answers and $P(a \mid \mathcal{K}_t)$ is the estimated probability of answer a given current knowledge.

Algorithm 1 Implemented spatial reasoning controller. The confidence score σ is a self-reported heuristic and is not calibrated.

Require: Task T , evidence $\mathcal{K}$ (fact sheet and spatial analysis), threshold $\tau = 0.6$

```

1:  $a \leftarrow \text{StrongModel}(T, \mathcal{K})$ 
2: if reflection is disabled then
3:   return  $a$ 
4: end if
5:  $\sigma \leftarrow \text{FastModel\_EstimateConfidence}(a, T, \mathcal{K})$ 
6: if  $\sigma < \tau$  then
7:    $a \leftarrow \text{StrongModel\_Refine}(T, a, \mathcal{K})$ 
8: end if
9: return  $a$ 

```

5.2 Action Selection via Information Gain

Given a set of candidate actions $\{c_1, \dots, c_m\}$ (e.g., examining a specific region of the image, querying the scene graph, calling a stronger model), we select the action that maximizes expected information gain:

$$c^* = \arg \max_{c_j} \mathbb{E} [H(\mathcal{A} \mid \mathcal{K}_t) - H(\mathcal{A} \mid \mathcal{K}_t \cup \text{obs}(c_j))] \quad (5)$$

In practice, we approximate this using self-reported model confidence. Each candidate answer a is accompanied by a score $\sigma(a) \in [0, 1]$ from a prompting heuristic. This score has not been demonstrated to be calibrated.

5.3 Reflection and Confidence Thresholds

The entropy-guided system triggers a *reflection* step when the confidence score falls below a threshold:

$$\text{reflect}(a) = \begin{cases} \text{True} & \text{if } \sigma(a) < \tau \\ \text{False} & \text{otherwise} \end{cases} \quad (6)$$

where $\tau = 0.6$ is the reflection threshold. During reflection, the agent re-examines its reasoning with additional context and issues one revised Strong-tier answer. The implemented path performs *at most one* such refinement per task. The configuration field named for reflection rounds acts as an enable flag rather than a round count, so a value greater than zero enables the single refinement and a value of zero disables it. The score σ is a self-reported heuristic and has not been shown to be calibrated, so it should not be read as a posterior probability.

5.4 Model Routing

A graded routing policy, in which a fast tier answers confidently handled questions, a standard tier handles moderate cases, and a strong tier is reserved for the hardest, is the design target for cost efficiency. We state it here as a target rather than as a description of the current spatial path.

The implemented spatial controller does not perform that graded escalation. It answers directly at the Strong tier over the already-constructed evidence, uses the fast tier only to produce a self-reported confidence score for that answer, and escalates no further than a single Strong-tier refinement. Consequently this paper makes no cost-efficiency claim for the spatial path. Whether graded routing reduces cost without degrading answers is an open question for the planned evaluation, and answering it requires the protected scoring that has not yet been performed.

Algorithm 1 states the procedure the spatial path actually executes.

6 Fail-Closed ML Pipeline

The MLE-Bench handler generates candidate pipelines from competition descriptions. Execution is fail closed: it requires both `ATLAS_ENABLE_MLEBENCH_CODE_EXECUTION=true` and `ATLAS_TRUSTED_ISOLATED_WORKER=true`. Server startup then requires `ATLAS_BEARER_TOKEN` containing at least 32 characters.

Competition Analysis: Upon receiving a competition task, the analyzer extracts structured meta-data including the task type, evaluation metric, data format, target column, and any special constraints. We classify competitions into six categories based on these features, as shown in Table 3.

Table 3: ML strategy templates and their target competition types.

Strategy	Task Type	Key Components
Tabular	Classification/Regression	LightGBM/XGBoost, feature engineering, cross-validation
NLP	Text Classification/NER	Transformer fine-tuning, TF-IDF fallback
Vision	Image Classification	Pre-trained CNN, transfer learning, augmentation
TimeSeries	Forecasting	Prophet, ARIMA, lag features, rolling statistics
General	Mixed/Unknown	Ensemble of lightweight models
AutoGluon	Any (fallback)	Time-limited AutoGluon TabularPredictor

6.1 Code Generation and Execution

For each competition, the pipeline generates a single standalone Python script that:

1. Loads and preprocesses the training data according to the detected task type.
2. Implements the selected strategy with appropriate hyperparameters.
3. Trains the model with cross-validation when the generated strategy supports it.
4. Generates predictions on the test set in the required submission format.
5. Writes a valid `submission.csv` to the expected output location.

After explicit authorization, the generated script runs in a bounded subprocess with a 600-second timeout. The subprocess captures bounded stdout and stderr, uses a minimal environment, and is terminated as a process group on timeout or cancellation. These controls are defense in depth, not a complete security sandbox.

Bounded Repair Loop: When an explicitly authorized execution fails, the repair mechanism may:

1. **Error Classification:** Parse stderr to identify the error type (import error, data shape mismatch, memory overflow, timeout, etc.).
2. **Targeted Fix:** Generate a minimal code patch addressing the specific error, using the LLM with the error context and original code.
3. **Re-execution:** Run the patched script with the same timeout constraints.

`max_code_iterations=3` means 3 total attempts, including the initial attempt. If all attempts fail, the task fails by default. A schema-shaped dummy submission is produced only when the operator separately sets `ATLAS_ALLOW_DUMMY_SUBMISSION=true`.

Score-Driven Refinement Loop: Error recovery alone cannot raise a working pipeline’s score. It only rescues pipelines that crash. After the first explicitly authorized run succeeds, the handler can parse a machine-readable line of the form `VALIDATION_SCORE: <float>`, request one targeted revision, re-run it under the same authorization and controls, and retain it only when the parsed score is better under the metric direction.

The loop runs up to `max_refinement_iterations=2` extra passes, bounded by a hard wall-clock ceiling (`refinement_wall_time_seconds=900`) to stay within MLE-Bench’s per-task budget. The selection logic is configured to discard revisions that regress or fail to print a score. This loop uses the configured Strong tier. Strong defaults to GPT-4.1 and can be overridden by the operator. Whether a cross-provider override outperforms a same-model retry is a planned ablation, not an established result.

6.2 Leak Audit and Targeted Leak Registry

The MLE-Bench paper and subsequent Kaggle post-mortems document a handful of competitions where the test set is reconstructable from training-set overlap, public dataset ancestry, or file metadata. Rather than hand-coding brittle exploit solvers (whose hard-coded merge keys may not match the MLE-Bench tar layout), Spatial Atlas maintains a *leak hint registry* whose entries are pure text instructions injected into the Strong-tier codegen prompt when a competition is detected.

Every codegen call also receives a universal *leak audit preamble* that instructs the Strong model to, before training any model:

1. Compare ID-like columns between train and test for row-level overlap.
2. Compute row fingerprints (hash of non-target features) to detect content duplication.

3. Check temporal ordering for timestamp-based competitions (train/test leakage through temporal shuffling).
4. Hash file bytes for media-based competitions to detect identical test/train files.

The audit fires independently of any registered entry, so an unregistered leak can still be surfaced when its exploit fits one of the four standard shapes. The audit’s detection rate has not been measured. Registered entries carry competition-specific detection predicates and targeted exploit sketches that take precedence over the generic audit. This design keeps the exploit code adaptive: the Strong model writes the final pandas operations against the actual tar layout it sees at runtime, while the audit policy itself remains auditable in a single file (`mlebench/strategies/leaks.py`).

Strategy Selection: The analyzer picks one strategy template from the task description. The ML path does not call the entropy engine, and it does not generate candidate solutions for several strategies. Entropy-guided strategy selection is not implemented.

7 Implementation Details

A2A Protocol Compliance: Spatial Atlas implements the A2A protocol specification using the official `a2a-sdk` (version $\geq 0.3.20$). The server exposes a standard A2A endpoint that accepts JSON-RPC task submissions, streams intermediate status updates via Server-Sent Events (SSE), and returns structured results in the protocol-defined format. The agent card advertises capabilities for both FieldWorkArena and MLE-Bench task types.

Deployment: The system is packaged as a Docker container targeting `linux/amd64`. Environment variables configure API keys, model endpoints, and resource limits. Public HTTP request bodies default to a 64 MiB limit. Active requests default to a maximum of 4, and excess requests receive HTTP 503 rather than entering an unbounded queue. `ATLAS_BEARER_TOKEN`, when configured, must contain at least 32 characters and authenticates non-read-only requests. Normal non-loopback startup requires the token even when MLE execution is disabled. Loopback development may omit it. `ATLAS_ALLOW_UNAUTHENTICATED_PUBLIC=true` is a test-only override for non-loopback binding and is not a normal deployment mode. MLE execution always requires authenticated startup.

File Processing Pipeline: Task inputs arrive in diverse formats requiring specialized processing:

- **Images:** JPEG/PNG files are processed through both GPT-4.1 vision (for scene description) and Florence-2 (for object detection and counting). Images are normalized to RGB and re-encoded before the API call. No resizing is applied.
- **PDFs:** Extracted using `pypdf` with page-by-page text extraction and optional OCR fallback.
- **Videos:** Frame extraction via OpenCV at one frame every two seconds, with an evenly spaced subset selected for analysis. No scene-change keyframe detection is performed.
- **Archives:** `tar.gz` files (MLE-Bench data) are extracted to a temporary workspace directory.
- **Text:** Direct UTF-8 processing with encoding detection fallback.

Model Configuration: All LLM calls use the model configurations specified in Table 1. The fast tier (`openai/gpt-4.1-mini`) handles entity extraction and self-confidence scoring. Domain classification uses deterministic rules and no model call at all. The standard and strong tiers both default to `openai/gpt-4.1`, and the strong tier can be overridden with `ATLAS_STRONG_MODEL`. Any cross-provider variant must be compared with the default under identical budgets.

Resource Controls: Public A2A model calls use the heuristic concurrency-safe per-execution reservation described above. Frozen benchmark drivers remain observational and use artifact-based batch accounting. Reflection performs at most one refinement per task. Generated-code execution is disabled by default and requires both execution flags plus authenticated server startup. After authorization, each pipeline attempt has a 600-second timeout and the initial plus repair loop permits at most 3 total attempts. Dummy submissions remain disabled unless separately enabled. After a successful real run, the score-driven refinement loop may execute up to 2 additional passes (Section 6), bounded by a 900-second wall-clock ceiling. Every number in this paragraph is a configured ceiling that bounds worst-case work. None is an observed runtime, an observed attempt count, or a measured cost.

8 Evaluation

8.1 Current Evidence Boundary

No claim-bearing benchmark table is reported in this version. FieldWorkArena remained gated and inaccessible, so the project did not run its validation set and reports no FieldWorkArena ac-

curacy, ablation, latency, token, or cost result. The local adapter and tests establish software behavior only. They are not benchmark evidence.

The repository also does not contain a sealed, end-to-end MLE-Bench result artifact covering the full competition suite. Accordingly, previously drafted valid-submission, medal, refinement, leak-effectiveness, and cost figures have been removed. A completed job or a working code path is not treated as a scientific result without the corresponding immutable predictions, scorer output, run manifest, and logs.

8.2 Label-Free Operational Validation

We report one completed label-free operational validation of the four-path comparison design. It is an integrity check on the execution path, not a measurement of answer quality.

Four paths were exercised together: the scene-graph path, a question-only baseline, a correct-image metric path, and a shuffled-image metric control. Each path produced exactly eight label-free prediction rows, for 32 rows in total. The run used a single content-pinned submission with zero retries and no rollback. Every bounded execution step completed successfully, all bounded service logs passed the required warning scan, registry restoration passed, the job-local authentication artifact was absent after teardown, and the terminal seal and terminal result were created exactly once.

Protected labels and ground truth were never opened. No benchmark score, paired comparison, uncertainty interval, cost result, or latency result was computed. The validation therefore establishes operational integrity only. It does not establish benchmark accuracy, superiority of any path, a causal benefit from correct metric geometry, generalization, calibration, or production readiness. In particular, the shuffled-image path is a control-path *execution*. Because the protected labels stayed sealed, its score and its paired effect against the correct-image path remain unknown.

Three distinctions matter for interpreting this result correctly. First, the prediction journals record model outputs before scoring. They are not answer-level evidence-use journals, and they do not establish which evidence a given answer actually used. Second, the terminal finalizer checks lifecycle and evidence-integrity conditions. It is not a post-kernel answer verifier and it does not check answer correctness. Third, the metric path exercised the bridge described in Section 4. No native persistent-kernel arm was executed.

Attribution is also worth stating plainly. The public repository supplies the metric bridge, the four run modes, the control-mapping contract, and the label-free journal validators. The combined single-root four-path orchestration that produced this validation was carried out in a separate private execution environment and is not part of the public repository.

8.3 Planned Evaluation Protocol

The spatial evaluation will compare a question-only baseline, the scene-graph path, the metric-perception path, and a native reference implementation on a frozen public slice. It will report per-question-type accuracy, paired uncertainty intervals, parser and geometry failure rates, model and data revisions, token usage, wall-clock latency, and artifact paths. Labels remain sealed until all prediction journals are complete. Protected scoring, paired analysis, and uncertainty analysis are the next scientific phase and have not been performed.

The ML-engineering evaluation will run fixed competition subsets with identical budgets and report valid-submission rate, competition-specific score, refinement acceptance rate, execution failures, tokens, latency, and cost. Each aggregate must be generated from machine-readable run artifacts rather than copied into the paper manually.

9 Discussion

Limitations: The boundaries of this work are extensive and we state them in full.

- **Estimated coordinates in the default path.** The default scene graph performs repeatable arithmetic over model-estimated two-dimensional coordinates. The arithmetic is deterministic, but its inputs are not measured, and deterministic arithmetic cannot correct an incorrect geometric input.
- **External perception dependencies.** The metric path requires an externally installed SpatialClaw package and a reachable GPU perception service. Neither is vendored, so the repository alone does not stand up that path.
- **Unbundled data and controls.** The gated benchmark images and the frozen shuffled-image control artifact are not distributed with the repository.
- **No combined orchestrator.** Public source provides the four run modes and their validators, not the single-root orchestration that produced the reported operational validation.

- **Inactive information-gain controller.** Expected-information-gain action selection is formulated here and present as a component, but the spatial path does not invoke it.
- **Uncalibrated confidence.** The self-reported confidence score gating refinement has not been shown to be calibrated and should not be read as a probability.
- **Unvalidated FieldWorkArena adapter.** The adapter was never run against the benchmark, whose data remained inaccessible, so it must not be presented as evaluated compatibility.
- **Proxy validation score.** The ML path parses a validation score emitted by generated code. That score is a proxy supplied by the pipeline under test, not an independently verified benchmark metric.
- **Incomplete executor isolation.** The execution controls are defense in depth and do not constitute a complete security sandbox.
- **Hand-designed templates and narrow leak coverage.** Strategy templates target common competition types and may not cover novel tasks, and the leak audit covers only four leakage shapes.
- **No cancellation.** Task cancellation is not supported once work is under way.
- **No native persistent loop and no answer-level verification.** There is no native integration of NVIDIA SpatialClaw’s persistent kernel, no journal recording which evidence each final answer used, and no post-generation verifier that checks an answer against captured evidence.
- **No scientific comparison.** No protected score, paired arm comparison, uncertainty interval, cost result, or latency result has been produced. Accuracy, superiority, calibration, generalization, and production readiness are all unestablished.

Future Work: Several directions for future work are promising:

- **Domain-Specific Fine-Tuning:** Fine-tuning Florence-2 on industrial environment imagery may improve object detection accuracy, particularly for domain-specific objects like safety equipment, pallet types, and industrial signage.
- **Multi-Agent Collaboration:** The A2A protocol enables multi-agent architectures where specialized sub-agents handle specific sub-tasks (e.g., one agent for visual analysis, another for spatial computation, a third for language generation).
- **Streaming Responses:** Implementing streaming A2A responses would enable real-time feedback during long-running ML pipeline executions.
- **Expanded Benchmarks:** Extending the architecture to additional benchmarks (e.g., SWE-Bench for software engineering, WebArena for web navigation) would test the generality of our approach.

Broader Impact: The spatial scene graph approach has direct applications to industrial safety, where automated monitoring of safety compliance (clearance distances, equipment placement, emergency exit accessibility) could prevent workplace injuries. However, automated spatial reasoning systems must be deployed carefully, with human oversight, as errors in safety-critical applications could have severe consequences.

10 Conclusion

We have presented Spatial Atlas, a spatial-aware research agent built on the compute-grounded reasoning (CGR) paradigm and exposed through an A2A protocol server. Our implemented contributions are:

1. A **spatial scene graph engine** that makes extracted entities, coordinate assumptions, and computed relationships inspectable without claiming that deterministic computation eliminates perception errors.
2. A **proposed entropy-guided reasoning framework** for targeted reflection. This repository does not wire it into model routing, and it reports no ablation of its effect.
3. A **fail-closed ML pipeline** with strategy-aware code generation, explicit execution authorization, bounded repair attempts, and a separately gated dummy fallback.
4. A **score-driven refinement loop** that parses validation scores, requests a revision from the configured strong tier, and retains it only when its parsed score improves.
5. A **leak audit registry** that prompts generated pipelines to check four common leakage patterns and can inject task-specific hints.
6. A **strict metric perception bridge** that replaces model-estimated coordinates with geometry reconstructed from segmentation masks and depth, records provenance and evidence digests, and fails closed rather than silently reverting to estimated coordinates.
7. **Label-free run modes and validators** for a four-path comparison, including a digest-bound, bijective, fixed-point-free shuffled-image control mapping and append-only journals that reject label-bearing content.

We also report one completed label-free operational validation in which all four paths executed and closed their bounded lifecycle, producing eight prediction rows per path and 32 rows in total, with protected labels and ground truth sealed throughout. That validation establishes operational integrity. It is not a benchmark result, and it does not rank the paths against one another.

Compute-grounded reasoning, the principle of computing what can be computed before generating what must be generated, offers a design pattern for making agent decisions more inspectable. Whether that structure improves accuracy, reliability, latency, or cost remains unanswered here. The next phase is the scientific one: unseal the protected labels under a frozen analysis plan, score the completed prediction journals, and report paired comparisons with uncertainty intervals. Only that step can convert the present operational evidence into a claim about performance.

The public code is at <https://github.com/arunshar/spatial-atlas-agent>. It includes the agent, the four run modes, and their validators, but it cannot re-execute the reported operational validation by itself (Appendix A).

A Reproducibility and Disclosure Boundary

Public entry points. The A2A server and task router are in `src/server.py` and `src/agent.py`. The spatial path is in `src/fieldwork/`, where `spatial.py` builds the scene graph, `perception.py` implements the strict metric bridge, and `reasoner.py` implements the controller of Algorithm 1. The ML path is in `src/mlebench/`. The benchmark driver exposing the run modes is `eval_bench.py`.

Run-mode definitions. A path is selected by two independent options. The engine is either `scenegraph` or `metric`. The image mode is one of `correct`, `question-only`, or `shuffled`. Four constraints hold. `question-only` requires the `scenegraph` engine, since no image is supplied to measure. `shuffled` requires the `metric` engine. `shuffled` requires an explicit control-mapping artifact. A control mapping is rejected for any image mode other than `shuffled`. The four paths reported in Section 8 are therefore `scenegraph+correct`, `scenegraph+question-only`, `metric+correct`, and `metric+shuffled`.

Control-mapping contract. A shuffled-image run is admitted only when the mapping is a bijection, contains no fixed point, and matches the recorded digest of every control image. Any drift, self-pairing, or non-bijective mapping is rejected before the run starts.

Label-free journal schema. Pre-scoring rows are appended under a versioned label-free schema that stores the model’s prediction text together with run metadata and no ground-truth field. The writer rejects label-bearing keys anywhere in a payload, refuses to mix label-free and ordinary rows in one journal, and refuses to resume a journal whose schema does not match the current run. These checks are what allow a run to complete and be inspected before any label is opened.

Test boundaries. The repository’s suites exercise the metric bridge, the run-mode selection and validators, the control-mapping contract, the journal schema, and the MLE execution gates. Passing tests establish the behavior they exercise. They are not benchmark evidence and they do not promote proposed behavior into a result.

What is not reproducible from this repository. The metric path requires an externally installed SpatialClaw package and a reachable GPU perception service. The gated benchmark images and the frozen control artifact are not bundled. The combined single-root orchestration used for the operational validation in Section 8 was performed in a separate private execution environment and is not included here. Consequently the reported operational validation cannot be re-executed from the public repository alone.

Disclosure boundary. This paper deliberately omits private execution identifiers, scheduler job identifiers, filesystem locations, credential material, service registry entries, raw service logs, and protected labels or ground truth. Their absence is a disclosure decision and not an omission of evidence relevant to any claim made here.

Planned protected scoring protocol. The scientific phase has not been performed. It is specified in advance as the following ordered steps.

1. Freeze the analysis plan and the interpretation boundary before anything is unsealed.
2. Verify that every prediction journal is complete and schema-valid.
3. Unseal the protected labels only after that verification passes.
4. Score each path with the benchmark’s own metric.
5. Report paired comparisons between the correct-image metric path and both the scene-graph path and the shuffled-image control, each with an uncertainty interval.

6. Report parser failures, geometry failures, refusals, and invalid cases alongside every aggregate. Every reported aggregate must be generated from machine-readable run artifacts rather than transcribed by hand.

References

- [1] Anthropic. Claude model family: Claude Opus 4.6 and Claude Sonnet 4.6. *Technical report*, 2025.
- [2] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, et al. SAM 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.
- [3] K. Chaloner and I. Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304, 1995.
- [4] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025. arXiv:2410.07095.
- [5] B. Chen, Z. Xu, S. Kirmani, et al. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [6] S. Cho, R. Hachiuma, A. Badki, H. Su, B.-K. Lee, C. H. Song, S. Liu, S. Radhakrishnan, S. Kim, Y.-C. F. Wang, and M.-H. Chen. SpatialClaw: Rethinking action interface for agentic spatial reasoning. *arXiv preprint*, 2026. <https://github.com/NVlabs/SpatialClaw>.
- [7] N. Erickson, J. Mueller, A. Shirkov, et al. AutoGluon-Tabular: Robust and accurate AutoML for structured data. *arXiv preprint arXiv:2003.06505*, 2020.
- [8] Matthias Feurer, Aaron Klein, Katharina Eggensperger, et al. Auto-sklearn 2.0: Hands-free AutoML via meta-learning. *Journal of Machine Learning Research*, 22(235):1–61, 2019.
- [9] J. Takahashi, A. Moteki, A. Uchida, S. Masui, F. Yang, K. Uchino, Y. Song, Y. Bisk, G. Neubig, I. Kusajima, Y. Watanabe, H. Ishida, K. Nakagawa, and S. Jiang. FieldWorkArena: Agentic AI benchmark for real field work tasks. *arXiv preprint arXiv:2505.19662*, 2025.
- [10] Google. Agent-to-Agent (A2A) protocol specification. Online documentation, 2024.
- [11] M. Hildebrandt, H. Li, R. Koner, et al. Scene graph reasoning for visual question answering. *arXiv preprint arXiv:2007.01072*, 2020.
- [12] Nikolaus Hollmann, Stefan Müller, and Frank Hutter. Large language models for automated machine learning. *arXiv preprint arXiv:2402.00878*, 2024.
- [13] S. Hong, X. Wang, J. Yu, et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [14] Drew Hudson and Christopher Manning. GQA: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.
- [15] Carlos Jimenez, John Yang, Ananya Wettig, et al. SWE-Bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- [16] Haifeng Jin, Qingquan Song, and Xia Hu. AutoKeras: An AutoML library for deep learning. *Journal of Machine Learning Research*, 24(6):1–6, 2023.
- [17] Ranjay Krishna, Yuke Zhu, Oliver Groth, et al. Visual Genome: Connecting language and vision using crowdsourced dense image annotations. *International Journal of Computer Vision*, 123:32–73, 2017.
- [18] Y. Li, Y. Du, K. Zhou, et al. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [19] Y.-H. Liao, R. Mahmood, S. Fidler, and D. Acuna. Reasoning paths with reference objects elicit quantitative spatial reasoning in large vision-language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. arXiv:2409.09788.
- [20] H. Lin, S. Chen, J. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang. Depth Anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.
- [21] BerriAI. LiteLLM: Call 100+ LLM APIs using the OpenAI format. GitHub repository, 2024.
- [22] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2024.
- [23] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [24] Burr Settles. Active learning literature survey. *Computer Sciences Technical Report 1648*, University of Wisconsin–Madison, 2009.
- [25] Significant Gravitas. AutoGPT: An autonomous GPT-4 experiment. GitHub repository, 2023.
- [26] Lei Wang, Chunping Ma, Xinyi Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):1–26, 2024.
- [27] B. Xiao, H. Wu, W. Xu, et al. Florence-2: Advancing a unified representation for a variety of vision tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [28] S. Xie, O. Levy, et al. Active prompting with chain-of-thought for large language models. *arXiv preprint arXiv:2302.12246*, 2024.
- [29] Danfei Xu, Yuke Zhu, Christopher Choy, and Li Fei-Fei. Scene graph generation by iterative message passing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

- [30] John Yang, Carlos Jimenez, Ananya Wettig, et al. SWE-Agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [31] Y. Zhang, H. Mao, Y. Zheng, et al. MLE-Agent: Automated machine learning engineering with LLM agents. *arXiv preprint arXiv:2402.15642*, 2024.